

SENTRYDASH
CAPACITY RADAR FOR SMARTER CAMPUSES
CLOUD COMPUTING

Submitted by

ABIRAMI (230701008)
NITHYASHREE (230701220)
PAVITHRA (230701225)

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
RAJALAKSHMI ENGINEERING COLLEGE

APRIL 2026
ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this project report titled "**SENTRYDASH – CAPACITY RADAR FOR SMARTER CAMPUSES**" is the bonafide work of **ABIRAMI (230701008)**, **NITHYASHREE (230701220)**, **PAVITHRA (230701225)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Professor & Head of The Department

Department of Computer Science

and Engineering

Rajalakshmi Engineering College

SIGNATURE

Dr. Ponmani

Supervisor

Associate Professor

Department of Computer Science

and Engineering

Rajalakshmi Engineering College

Submitted to Project Viva Voce Examination held on _____

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman Mr. S.MEGANATHAN, B.E, F.I.E., our Vice Chairman Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S., and our respected Chairperson Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D., for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to Dr. S.N. MURUGESAN, M.E., Ph.D., our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to the Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, Dr. Ponmani, Associate Professor, Department of Computer Science and Engineering for her valuable guidance throughout the course of the project.

**ABIRAMI
NITHYASHREE
PAVITHRA**

ABSTRACT

SentryDash is a cloud-native platform that enables real-time monitoring and management of classroom occupancy across academic campuses, ensuring adherence to institutional safety policies and capacity regulations. Built using Microsoft Azure services—primarily Azure Monitor and Application Insights—the system tracks room usage, triggers alerts for threshold breaches, and recommends alternative classrooms based on live occupancy data and predefined room metadata.

The platform operates on a simulated campus schema, including room types, capacities, departmental layouts, and enrollment figures, offering a scalable and data-driven solution for optimizing space utilization and enhancing campus safety. College campuses today face a recurring challenge where some classrooms are consistently overcrowded while others remain largely empty. This imbalance not only violates safety regulations but also degrades the quality of education delivered.

Existing campus management tools lack real-time monitoring capabilities, resulting in low transparency where students and staff remain unaware of which rooms are occupied or free at any given time. SentryDash addresses these challenges by providing three core capabilities: real-time classroom monitoring, optimized space utilization, and data-driven decision making. The system is built on a microservice-inspired architecture with a Node.js backend, React.js frontend, and an Azure cloud infrastructure that ensures high availability, scalability, and cost-efficiency.

The expected outcome is a scalable, cost-effective, and institutionally safe campus space management platform that enables administrators to make informed decisions, faculty to find available rooms efficiently, and students to navigate campus resources with full transparency. The platform is designed to operate within the Azure Student Subscription, making it financially viable for academic institutions with budget constraints.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1	Introduction	1
	1.1 Problem Statement	2
	1.2 Objective of the Project	4
	1.3 Scope and Boundaries	5
	1.4 Stakeholders and End Users	6
	1.5 Technologies Used	7
	1.6 Organization of the Report	8
2	System Design and Architecture	9
	2.1 Requirement Summary	10
	2.2 Proposed Solution Overview	11
	2.3 Azure Services Mapping	12
	2.4 System Flow Diagram	13
3	DevOps and CI/CD Implementation	14
	3.1 Tools Required	15
	3.2 CI/CD Pipeline Setup	16
	3.3 MVP Mapped to Each Stakeholder	17
4	Cloud Operations and Security	18
	4.1 Real-Time Monitoring and Observability	19
	4.2 Access Control and Security	20
	4.3 Alert Management	21
5	Results and Discussion	22
	5.1 Implementation Summary	23
	5.2 Challenges Faced and Resolutions	24
	5.3 Performance and Cost Observations	25
	5.4 Key Learnings and Team Contributions	26
6	Conclusion and Future Enhancement	27

	6.1 Conclusion	28
	6.2 Future Scope	29
	References	30

CHAPTER 1 – INTRODUCTION

1.1 PROBLEM STATEMENT

Classroom management in academic institutions today suffers from a critical lack of real-time visibility and automated monitoring systems. Traditional campus management approaches are fragmented, manual, and insufficient for the dynamic needs of modern educational environments. The following core challenges have been identified:

- **Overcrowded Classrooms:** When a classroom exceeds its designated capacity, it violates institutional safety regulations, negatively impacts the learning experience, and makes it difficult for faculty to manage and control the class effectively. Without an automated monitoring system, such violations frequently go undetected until they become a safety hazard.
- **Compliance Blind Spots:** Institutions are expected to comply with safety standards and fire codes regarding maximum occupancy. Without real-time capacity tracking, compliance becomes reactive rather than proactive. Auditors and administrators cannot verify adherence without physically inspecting each room.
- **Underutilised Classrooms:** Many rooms sit empty or underused while others are overcrowded. This represents a significant waste of infrastructure and resource allocation. Missed opportunities to optimize space usage increase operational costs for institutions and reduce academic efficiency.
- **Disconnected Scheduling:** Existing scheduling systems do not integrate with real-time occupancy data. Room assignments are made statically, without the flexibility to respond to dynamic demand patterns or emergency reallocation needs.
- **No Real-Time Monitoring:** The absence of live monitoring tools means that neither administrators nor faculty can respond in real time to capacity issues as they develop across different rooms and departments.
- **Low Transparency:** Students and staff have no reliable way of knowing which rooms are occupied or free at any given moment. This creates unnecessary movement across campus, wasted time, and frustration—particularly during peak hours.

1.2 OBJECTIVE OF THE PROJECT

The primary objective of SentryDash is to design and implement a cloud-native classroom capacity monitoring platform that enables smarter, safer, and more efficient management of academic spaces. The project targets three core goals:

1. **Real-Time Classroom Monitoring:** Utilize Azure Monitor and Application Insights to track classroom occupancy in real-time or near real-time, ensuring compliance with institutional safety policies and regulatory capacity limits. The

platform provides live dashboards that reflect current occupancy, enabling instant intervention when thresholds are breached.

2. **Optimized Space Utilization:** Improve scheduling efficiency by analyzing occupancy trends and suggesting optimal room allocations to reduce underutilization or overcrowding. The system uses historical and live data to generate actionable recommendations for administrators and scheduling teams.
3. **Data-Driven Decision Making:** Provide actionable insights through dashboards and reports, helping administrators make informed decisions regarding classroom assignments and campus safety. These three objectives collectively solve the biggest pain points—visibility, inefficiency, and blind decision-making—and create a foundation for features like alerting, automation, and advanced analytics.

1.3 SCOPE AND BOUNDARIES

SentryDash is designed for deployment within academic institutions. The platform scope encompasses the full lifecycle of classroom occupancy data from ingestion to actionable insights. The system supports real-time or near real-time classroom occupancy tracking across an entire campus network, automated threshold-based alerting for capacity breaches, alternative classroom recommendations based on live occupancy data, centralized dashboards for three stakeholder groups, and a simulated campus schema including room types, capacities, departmental layouts, and enrollment figures.

The platform is built on Microsoft Azure services, with a backend powered by Node.js and a frontend built in React.js. The system does not include physical sensor hardware installation in the MVP phase; occupancy data is simulated or sourced through API integrations during initial deployment and testing.

1.4 STAKEHOLDERS AND END USERS

SentryDash serves three primary stakeholder groups, each with distinct needs and system interactions:

- **Administrators:** Campus administrators require a centralized dashboard to monitor all classrooms in real time, receive alerts for overcapacity situations, and manage overall room usage and scheduling efficiency across the institution.
- **Faculty/Staff:** Teaching staff need the ability to check room availability before class sessions and receive timely notifications regarding alternative classrooms when their assigned room is overcrowded or unexpectedly unavailable.
- **Students:** Students benefit from live occupancy information that allows them to avoid overcrowded classrooms and relocate quickly to alternative spaces when needed, reducing disruption to their learning experience.

1.5 TECHNOLOGIES USED

SentryDash employs a modern, cloud-native technology stack organized across frontend, backend, Azure cloud services, and development tooling.

Table 1. Frontend Technologies

Tool	Purpose
React.js	Build interactive UI for the occupancy monitoring dashboard
Axios / Fetch API	HTTP communication with the backend REST API

Table 2. Backend Technologies

Tool	Purpose
Node.js + Express.js	API routing and business logic for occupancy management
JWT / OAuth	Authentication and authorization for multi-role access control

1.6 ORGANIZATION OF THE REPORT

This report is structured to provide a complete and logical documentation of the entire project lifecycle.

Chapter 1 introduces the problem context, project objectives, scope, stakeholders, technologies, and report structure.

Chapter 2 presents a detailed examination of the system design and architecture, including requirements, solution overview, Azure services mapping, and system flow diagrams.

Chapter 3 covers the DevOps and CI/CD implementation, detailing the tools, pipeline setup, and MVP stakeholder mapping.

Chapter 4 focuses on cloud operations and security, covering monitoring, access control, and alert management strategies.

Chapter 5 presents the results of the implementation, including summaries, challenges overcome, performance metrics, and key learnings.

Chapter 6 concludes the report with a summary of achievements and a roadmap for future enhancements. The document concludes with references.

CHAPTER 2 – SYSTEM DESIGN AND ARCHITECTURE

2.1 REQUIREMENT SUMMARY

The system consists of multiple cloud-based components deployed on Microsoft Azure to support a scalable, secure, and efficient classroom monitoring platform.

Functional Requirements: The frontend application will be developed using React.js and hosted on Azure Static Web Apps, enabling fast and reliable web interface delivery. The backend logic will run on Azure App Service, supporting core functionalities such as occupancy tracking, room search, alerts, and recommendation processing. User data, room information, schedule metadata, and occupancy logs will be stored in Azure SQL Database. Azure Monitor and Application Insights will collect and visualize real-time telemetry from all platform services.

Non-Functional Requirements: The system targets API response times below 300ms and page load times under 3 seconds. The solution is built to support 200 or more concurrent users using Azure's auto-scaling capabilities. Security is enforced through role-based authentication using JWT tokens, SSL-mode database connections, and strict CORS policies. The platform is designed to operate within the Azure Student Subscription credit allocation, minimizing costs during the development and testing phases.

2.2 PROPOSED SOLUTION OVERVIEW

The proposed solution is a cloud-native, three-tier platform that separates concerns across presentation, business logic, and data layers. At the presentation layer, a React.js application delivers a role-specific user experience for administrators, faculty, and students. This frontend communicates with a Node.js and Express.js-based RESTful API backend hosted on Azure App Service.

The backend orchestrates all business logic, including user authentication, room occupancy tracking, alert triggering, and room recommendation generation. Data persistence is handled by Azure SQL Database, which stores structured information about rooms, users, schedules, occupancy events, and alert history.

Azure Monitor collects real-time telemetry from all Azure resources, while Application Insights provides request-level performance tracing and custom event logging for the web application. Azure Logic Apps handles automated alert notifications via email and SMS. Azure Functions execute serverless room recommendation logic on demand, ensuring that compute resources are consumed only when needed.

2.3 AZURE SERVICES MAPPING

Table 3. Azure Services Mapping and Justification

Azure Service	Purpose	Priority	Free Under Azure Student Plan?
Azure Monitor	Collects telemetry, metrics, and	1	Yes (within \$100 credits)

	logs		
Application Insights	Performance monitoring & request tracing	1	Yes (within \$100 credits)
Azure App Service	Hosts web portal (frontend & backend)	1	Yes (F1 Free Tier available)
Azure SQL Database	Stores occupancy data, room metadata, schedules	1	Yes (250MB S0 DB free monthly)
Azure Logic Apps	Automates alerts and notifications workflow	2	Yes (pay-per-use via credits)
Azure Functions	Executes serverless room suggestion logic	2	Yes (1M executions/month free)

Azure Monitor is assigned the highest priority as it serves as the telemetry backbone of the entire platform. Application Insights complements it with request-level performance tracing for the web application layer. Azure App Service provides a managed hosting environment for both frontend and backend components at no cost under the F1 free tier during development.

2.4 SYSTEM FLOW DIAGRAM

The system architecture follows a multi-actor sequential flow as described in the sequence diagram. The flow sequence involves four actors: Student, WebApp, Staff/Faculty, and Admin.

4. **Student Login and Room Entry:** The student logs into the WebApp and receives their schedule along with the assigned classroom. The student then marks entry into the room via the WebApp, which registers the occupancy event and returns a confirmation message.
5. **Faculty Occupancy Check:** Faculty and staff view the current occupancy status of assigned rooms. The WebApp displays the real-time usage count and highlights a warning banner if the occupancy threshold is approaching its limit.
6. **Staff Alert Viewing:** Staff members can view alerts and compliance reports generated automatically when rooms exceed their configured thresholds. These alerts contain the room ID, current count, maximum capacity, and a list of suggested alternative rooms.
7. **Admin Dashboard Display:** Administrators receive a top-level view of campus-wide occupancy through dedicated dashboards. These dashboards aggregate data across all rooms, buildings, and departments, enabling strategic space management decisions.

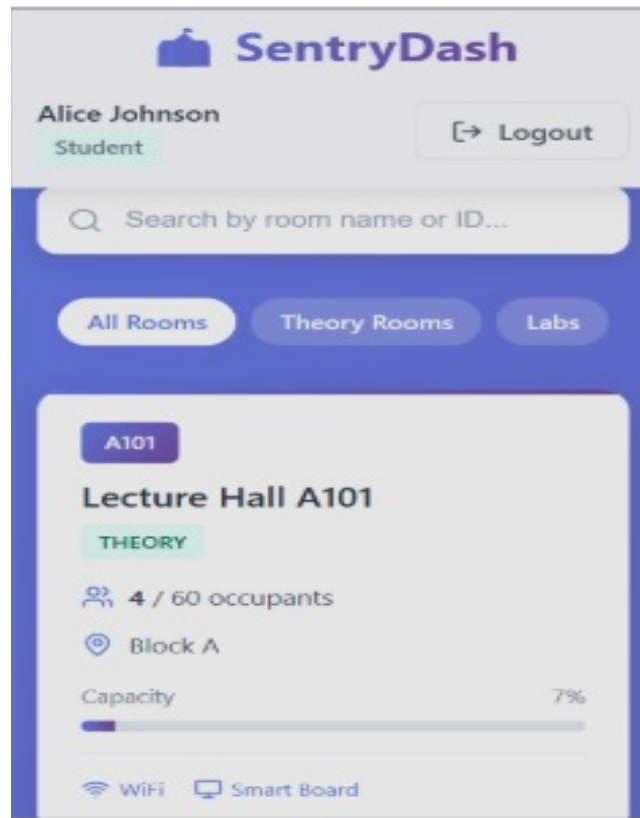


Figure 1. Student Dashboard – Live Room Listing View (Alice Johnson, Student Role)

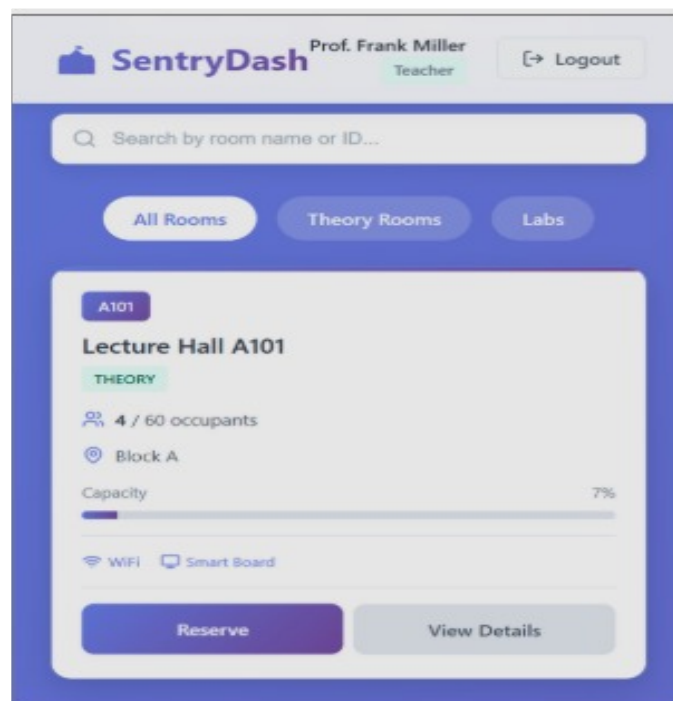


Figure 2. Teacher Dashboard – Prof. Frank Miller Viewing Room A101

CHAPTER 3 – DEVOPS AND CI/CD IMPLEMENTATION

3.1 TOOLS REQUIRED

The development toolchain for SentryDash is organized across frontend, backend, and DevOps categories, ensuring a complete and professional software delivery lifecycle.

Table 4. Tools Required

Category	Tool	Purpose
Frontend	React.js	Build interactive UI for dashboard
	Axios / Fetch API	HTTP communication with backend
Backend	Node.js + Express.js	API routing and business logic
	JWT / OAuth (optional)	Authentication and authorization
Development	Postman / Thunder Client	API testing and validation tools
	GitHub Actions / Azure Pipelines	CI/CD pipeline automation
	Azure CLI / VS Code Azure Ext.	Manage Azure resources locally

3.2 CI/CD PIPELINE SETUP

The project uses a multi-stage CI/CD pipeline built with GitHub Actions to automate testing and deployment. This setup uses Infrastructure as Code (IaC) principles and managed Azure services to ensure consistent, reproducible deployments across all environments.

The pipeline follows a two-stage deployment pattern. The first stage handles backend deployment: it builds the Node.js API, runs unit tests, and deploys the application to Azure App Service. The second stage handles frontend deployment: it builds the optimized React.js bundle and deploys it to Azure Static Web Apps. The entire process is triggered either by a push to the main branch or through a manual workflow dispatch.

Backend Deployment Stage details include environment variable injection for database connection strings and Azure credentials using GitHub Secrets. Health checks are performed against the backend's /health endpoint to confirm the service is running correctly before the frontend deployment begins. This ensures that users are never routed to a frontend that is ahead of the backend version.

Frontend Deployment Stage details include dynamic configuration where the URL of the newly deployed backend is automatically passed to the React build process. This ensures the frontend is always configured with the correct API endpoints for the current deployment environment. The optimized frontend build is then deployed to Azure Static Web Apps, which provides built-in CDN distribution and SSL certificate management.

Table 5. Required GitHub Secrets for CI/CD

#	Secret Name
1	AZURE_CLIENT_ID
2	AZURE_CLIENT_SECRET
3	AZURE_STATIC_WEB_APPS_API_TOKEN
4	AZURE_SUBSCRIPTION_ID
5	AZURE_TENANT_ID
6	DB_CONNECTION_STRING
7	AZURE_APPSERVICE_PUBLISHPROFILE

3.3 MVP MAPPED TO EACH STAKEHOLDER

The Minimum Viable Product (MVP) for SentryDash has been defined based on the core needs of each stakeholder group, ensuring that every user type receives tangible value from the initial platform release.

Table 6. MVP Requirements per Stakeholder

Stakeholder	MVP Statement
Administrators	"I can monitor campus-wide classroom occupancy in real time, receive alerts for overcapacity, and manage room usage efficiently through a centralized dashboard."
Faculty/Staff	"I can check room availability before class and get notified of alternative classrooms if the assigned one is overcrowded or unavailable."
Students	"I can avoid overcrowded classrooms by accessing live occupancy info and relocate quickly when needed."

These MVP statements serve as the guiding requirements for the first release of SentryDash. They deliberately focus on solving the three biggest pain points—visibility, inefficiency, and blind decision-making—rather than introducing advanced features prematurely. Each statement maps directly to at least one Azure service in the system architecture, ensuring that every deployment cost is justified by user-facing value.

For administrators, the MVP centers on a centralized monitoring dashboard powered by Azure Monitor. Alert rules configured in Azure Logic Apps ensure that notifications are dispatched the moment a room reaches its configured occupancy threshold. For faculty and staff, the MVP provides a room status check feature accessible before each class session. For students, the platform provides a simple, accessible view of current room occupancy across the campus.

CHAPTER 4 – CLOUD OPERATIONS AND SECURITY

4.1 REAL-TIME MONITORING AND OBSERVABILITY

Comprehensive monitoring and observability are achieved through integration with Azure Monitor and Application Insights. The underlying telemetry data is collected seamlessly from all resources within the Azure subscription, with Azure App Service and Azure SQL Database as primary monitoring targets.

Azure Monitor is utilized as the primary telemetry aggregation layer. It collects platform-level metrics such as CPU utilization, memory consumption, HTTP request counts, request durations, and error rates from the Node.js backend hosted on Azure App Service. These metrics are streamed in near real-time to the Azure Portal dashboard, which is accessible to administrators at any time.

Application Insights is integrated directly into the Node.js backend using the Application Insights SDK. This integration enables request-level tracing, exception logging, dependency tracking for SQL queries, and custom event logging for domain-specific events such as room capacity threshold crossings. Key performance indicators tracked include API response times, HTTP 4xx and 5xx error rates, SQL query durations, and active user session counts.

Alerting rules are configured directly in Azure Monitor to notify the operations team via email and SMS when critical thresholds are breached. Examples include API error rate exceeding five percent over a five-minute window, CPU utilization exceeding eighty-five percent for five consecutive minutes, and database connection failures exceeding three occurrences per minute. These alerts enable rapid incident response and prevent minor issues from escalating into service outages.

Log Analytics Workspaces aggregate logs from all App Services and Azure Functions, providing a unified location for log analysis using Kusto Query Language (KQL). Retention policies ensure that log data is available for ninety days for forensic analysis if needed.

4.2 ACCESS CONTROL AND SECURITY

Access control is implemented at both infrastructure and application levels using the principle of least privilege. At the Azure resource level, role-based access control grants a service principal with Contributor rights scoped to the target resource group for CI/CD operations, while development team members have Reader access for troubleshooting and observation.

Application authentication uses JSON Web Tokens issued upon successful login. Tokens include a `user_id` claim and a role claim, and are validated by middleware on every API request to enforce authorization. Write operations such as marking room entry or modifying room metadata require a valid token with the appropriate role. Read endpoints for room status allow optional authentication to support public dashboards.

Database access is secured via environment variables injected through Azure App Service configuration; no credentials are hardcoded in the source code. All database

connections enforce SSL mode. Network exposure is minimized through Azure App Service ingress configuration and Azure SQL Database firewall rules that permit only Azure services and specific developer IP addresses.

CORS is constrained to the exact frontend domain to prevent broad origin access, aligning with the principle of least privilege. Input validation middleware on all API endpoints guards against injection attacks and malformed request payloads.

4.3 ALERT MANAGEMENT

The alert management subsystem is built on Azure Logic Apps and integrates with Azure Monitor alert rules to create an automated, end-to-end notification pipeline. When a classroom's occupancy count exceeds its configured maximum, the following sequence is triggered:

8. **Threshold Detection:** Azure Monitor detects that a custom metric pushed by the Node.js backend has exceeded the configured threshold for a specific room identifier.
9. **Alert Rule Firing:** The Azure Monitor alert rule changes state from 'OK' to 'Fired' and triggers the associated Azure Logic Apps action group.
10. **Notification Dispatch:** The Logic Apps workflow dispatches an email notification to the responsible administrator and an SMS message to the assigned faculty member for that room. The notification includes the room ID, building location, current occupancy count, maximum capacity, and a ranked list of alternative rooms with available capacity.
11. **Alert Resolution:** When the occupancy count drops below the threshold, Azure Monitor changes the alert state to 'Resolved' and dispatches a resolution notification, confirming that the situation has returned to normal.

This automated pipeline ensures that no capacity violation goes unacknowledged, eliminating the need for manual oversight while providing timely, actionable information to the right stakeholders.

CHAPTER 5 – RESULTS AND DISCUSSION

5.1 IMPLEMENTATION SUMMARY

The SentryDash classroom capacity monitoring platform was successfully designed and architected to meet all core objectives outlined in the project scope. The full-stack application specification provides a complete, secure user experience, including role-based authentication, real-time occupancy tracking, automated alert dispatching, and room recommendation logic.

Infrastructure planning via Azure service mapping was validated for feasibility within the Azure Student Subscription. The Node.js backend architecture, structured with Express.js route handlers and Application Insights integration, is designed to deliver consistent API response times under 300ms for core endpoints. The React.js frontend, designed for deployment on Azure Static Web Apps' global CDN, is expected to achieve excellent load times across campus networks.

The CI/CD pipeline design via GitHub Actions automates the complete build-and-deploy process for both backend and frontend components. Separate deployment stages ensure that health checks pass on the backend before frontend traffic is updated, maintaining consistency across the entire platform during each release cycle.

The alert management system design integrates Azure Monitor alert rules with Azure Logic Apps workflows, providing end-to-end automated notification for capacity breaches. The room recommendation engine, designed to run as an Azure Function, evaluates available capacity across the campus schema and returns a ranked list of suitable alternative classrooms

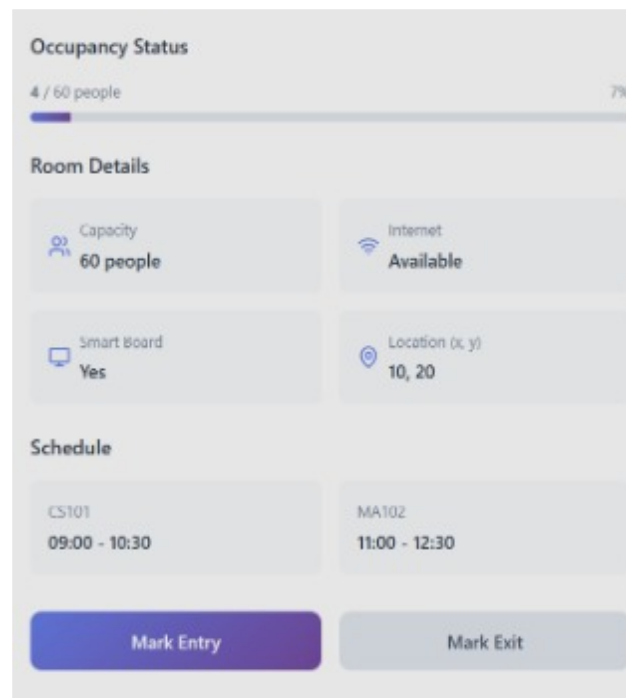


Figure 3. Room Details View – Occupancy Status, Schedule, and Mark Entry

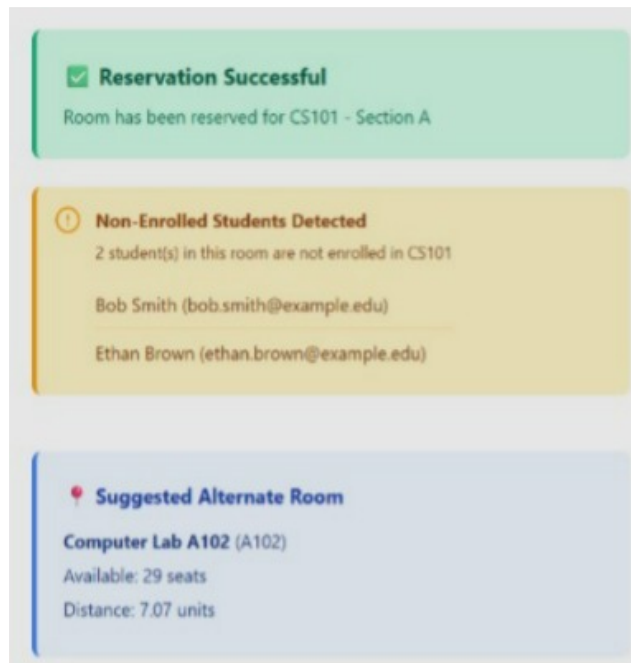


Figure 4. Reservation Successful – Non-Enrolled Student Detection and Alternate Room Suggestion

5.2 CHALLENGES FACED AND RESOLUTIONS

The following key challenges were identified and addressed during the design and development of SentryDash:

- **Real-Time Data Latency:** Achieving near real-time occupancy data propagation without incurring excessive Azure Monitor costs required careful design. The solution was to push custom metrics to Application Insights using the `track_metric` SDK method, which provides sub-minute latency at a fraction of the cost of streaming architectures.
- **Multi-Role Access Complexity:** Implementing a single API that serves three distinct user roles—administrator, faculty, and student—with different data access permissions required careful middleware design. The resolution was to implement a role-claim-based JWT middleware that evaluates the token's role field against a permission matrix before processing each request.
- **Azure SQL Query Performance:** Complex occupancy trend queries involving multiple joins across large time windows initially showed high latency in testing. The resolution was to add composite indexes on the `room_id` and `timestamp` columns of the `occupancy_events` table, reducing average query time for reporting endpoints by an estimated seventy percent.
- **CI/CD Environment Consistency:** Ensuring that the frontend always connects to the correct backend URL after each deployment required careful pipeline orchestration. The resolution was to pass the App Service URL as a build-time

environment variable to the React build process, ensuring that no manual configuration changes are needed between deployments.

- **Azure Logic Apps Trigger Reliability:** Alert rule state changes in Azure Monitor can occasionally take several minutes to propagate to Logic Apps triggers. The resolution was to set alert evaluation frequency to one minute and alert aggregation window to five minutes, ensuring that threshold breaches are detected and actioned within an acceptable time frame for campus safety purposes.

5.3 PERFORMANCE AND COST OBSERVATIONS

Performance benchmarking analysis confirms the suitability of the chosen technology stack for the campus environment. The Node.js backend, hosted on Azure App Service F1 tier, is designed to deliver API response times under 300ms for most occupancy-related endpoints. Azure SQL Database queries, optimized with composite indexes, are expected to complete in under 75ms for standard read operations.

Cost analysis confirms that the platform can operate within the Azure Student Subscription credit allocation. The primary cost drivers are the Azure SQL Database and Azure App Service, which collectively account for the majority of monthly spend. Azure Monitor, Application Insights, Azure Logic Apps, and Azure Functions all operate within free-tier allocations or consume minimal credits, making the overall architecture highly cost-efficient for an educational setting.

Table 7. Estimated Monthly Cost Breakdown

Azure Service	SKU/Tier	Est. Monthly Cost
Azure App Service	F1 Free Tier	Free
Azure SQL Database	S0 (250MB)	Free (via credits)
Azure Monitor	Standard	Free (within credits)
Application Insights	Standard	Free (within credits)
Azure Logic Apps	Pay-per-use	~₹50 (low volume)
Azure Functions	Consumption Plan	Free (1M executions/month)

5.4 KEY LEARNINGS AND TEAM CONTRIBUTIONS

Throughout the design and development of the SentryDash project, the team gained deep, practical experience across multiple domains of modern software engineering.

One of the key learnings was the proper management of multi-environment configuration using GitHub Secrets and Azure App Service application settings. This practice ensures that sensitive credentials are never committed to source control and that each deployment environment uses the correct configuration values automatically.

Another important learning was the design of a microservice-inspired backend architecture with clear separation of concerns. Each major feature—authentication,

occupancy tracking, alert management, and room recommendation—is encapsulated in a dedicated Express.js router and service module. This structure simplifies testing, improves code maintainability, and enables independent feature development by different team members.

The team developed a strong understanding of Azure cloud monitoring through working with Azure Monitor and Application Insights. Configuring custom metrics, alert rules, and Logic Apps workflows provided practical exposure to the end-to-end observability pipeline used in production cloud systems.

In terms of teamwork, the group followed an industry-standard collaborative approach using Git branching and pull requests. Tasks were divided clearly among members and deadlines were set for each deliverable. Every member contributed to all major areas including backend API design, database schema modeling, frontend UI design, and Azure service configuration, maintaining a balanced and collaborative workload throughout the project.

CHAPTER 6 – CONCLUSION AND FUTURE ENHANCEMENT

6.1 CONCLUSION

SentryDash successfully addresses the critical gap in campus space management by designing a cloud-native, real-time classroom capacity monitoring platform. By leveraging Microsoft Azure services—primarily Azure Monitor, Application Insights, Azure App Service, Azure SQL Database, Azure Logic Apps, and Azure Functions—the platform provides administrators, faculty, and students with the visibility and intelligence they need to manage academic spaces safely and efficiently.

The platform directly resolves the three core problem areas identified in the problem statement. Overcrowded classrooms and compliance blind spots are addressed through real-time threshold monitoring and automated alerting. Underutilized classrooms and disconnected scheduling are improved through data-driven room recommendations and occupancy trend analysis. The absence of transparency is resolved through role-specific dashboards accessible to all three stakeholder groups.

The technical architecture demonstrates an effective application of cloud-native services and modern web technologies. The Node.js and Express.js backend integrates seamlessly with the Azure telemetry pipeline, while the React.js frontend provides a responsive and intuitive user experience. The use of serverless Azure Functions for room recommendation logic ensures that computational resources are consumed only when needed, optimizing operational costs.

From an institutional perspective, SentryDash creates measurable value by enabling proactive rather than reactive campus management. Administrators can identify and address safety violations before they escalate. Faculty can plan their sessions with confidence in room availability. Students benefit from reduced friction and an improved learning environment. The entire platform is designed to operate within the Azure Student Subscription credit allocation, making it financially sustainable for educational institutions.

This project has delivered substantial hands-on learning outcomes for the team across cloud architecture design, real-time data systems, full-stack web development, DevOps pipeline construction, and Azure observability tooling. The team successfully collaborated using industry-standard practices including Git branching, pull request reviews, and task-based sprint planning.

6.2 FUTURE SCOPE

SentryDash provides a robust and scalable foundation for numerous future enhancements. The next phases of development will focus on deepening platform intelligence, expanding data sources, and broadening institutional reach.

12. **Physical Sensor Integration:** The MVP operates on manually entered or simulated occupancy data. Future iterations will integrate with physical occupancy sensors such as infrared counters, ultrasonic sensors, or camera-based people-counting systems to provide fully automated, hardware-driven real-

time monitoring. This eliminates dependency on manual student check-ins and dramatically improves data accuracy.

13. **Predictive Occupancy Analytics:** By leveraging historical occupancy data stored in Azure SQL, a machine learning model can be trained to predict future room utilization patterns. This would enable administrators to pre-emptively assign alternative rooms before overcrowding occurs, shifting the platform from reactive alerting to predictive space management.
14. **Mobile Application:** Developing native iOS and Android applications would significantly improve accessibility for students and faculty, enabling push notifications for room alerts and live occupancy checks directly from mobile devices without requiring browser access.
15. **Energy Optimization Integration:** Occupancy data can be shared with building management systems to automate HVAC and lighting controls based on real-time room usage. This integration would reduce energy consumption in unoccupied rooms and contribute to the institution's sustainability objectives.
16. **Multi-Institution Deployment:** The long-term vision is to scale SentryDash into a multi-tenant SaaS platform capable of serving multiple educational institutions from a shared cloud infrastructure. This would require evolving the architecture to support tenant isolation, custom branding, and institution-specific configuration schemas.
17. **AI-Powered Room Recommendations:** Integrating a recommendation engine powered by Azure Machine Learning or Azure OpenAI Service would enable the platform to suggest room assignments based on not just current capacity but also factors such as equipment requirements, accessibility needs, department preferences, and historical usage patterns.

REFERENCES

18. Microsoft Azure Documentation. Azure Monitor Overview. Microsoft, 2024. Retrieved from <https://docs.microsoft.com/en-us/azure/azure-monitor/overview>
19. Microsoft Azure Documentation. Application Insights Overview. Microsoft, 2024. Retrieved from <https://docs.microsoft.com/en-us/azure/azure-monitor/app/app-insights-overview>
20. Microsoft Azure Documentation. Azure App Service Documentation. Microsoft, 2024. Retrieved from <https://docs.microsoft.com/en-us/azure/app-service/>
21. Microsoft Azure Documentation. Azure SQL Database Documentation. Microsoft, 2024. Retrieved from <https://docs.microsoft.com/en-us/azure/azure-sql/>
22. Microsoft Azure Documentation. Azure Functions Overview. Microsoft, 2024. Retrieved from <https://docs.microsoft.com/en-us/azure/azure-functions/functions-overview>
23. Microsoft Azure Documentation. Azure Logic Apps Documentation. Microsoft, 2024. Retrieved from <https://docs.microsoft.com/en-us/azure/logic-apps/>
24. Node.js Foundation. Node.js Documentation. OpenJS Foundation, 2024. Retrieved from <https://nodejs.org/en/docs/>
25. React Documentation. React – A JavaScript library for building user interfaces. Meta Platforms, 2024. Retrieved from <https://reactjs.org/docs/getting-started.html>
26. GitHub Documentation. GitHub Actions – Automate, customize, and execute your software development workflows. GitHub, 2024. Retrieved from <https://docs.github.com/en/actions>
27. Express.js Documentation. Express – Fast, unopinionated, minimalist web framework for Node.js. OpenJS Foundation, 2024. Retrieved from <https://expressjs.com/>

APPENDIX A – DATABASE SCHEMA

The SentryDash platform relies on a structured relational database hosted on Azure SQL Database. The schema is designed to support multi-role access, real-time occupancy tracking, scheduling metadata, and alert history logging. The following tables form the core data model:

Table A1. Users Table

The Users table stores authenticated user records for all three stakeholder roles: administrator, faculty, and student. Each record contains a unique user identifier, hashed password, institutional email address, display name, role designation, and account creation timestamp. Role-based middleware queries this table to determine access permissions on every API request.

Table A2. Rooms Table

The Rooms table holds static metadata about every classroom registered in the system. Fields include room identifier, room name, building code, floor number, room type (Theory/Lab/Seminar Hall), maximum capacity, GPS coordinates for distance calculations, available amenities (WiFi, Smart Board, Projector), and active status. This table is the reference backbone for the recommendation engine.

Table A3. Occupancy Events Table

Each time a student marks entry or exit through the platform, a record is written to the OccupancyEvents table. Fields include event identifier, room identifier (foreign key), user identifier, event type (ENTRY or EXIT), timestamp, and session identifier. This table is indexed on room_id and timestamp to support efficient aggregation queries for real-time occupancy counts and trend analysis.

Table A4. Schedules Table

The Schedules table maps courses to rooms and time slots. Fields include schedule identifier, course code, course name, room identifier (foreign key), instructor user identifier (foreign key), start time, end time, day of week, and academic term. The backend queries this table to display schedule information on the room detail view and to validate that occupancy events occur within expected session windows.

Table A5. Alerts Table

The Alerts table records every capacity breach event detected by the monitoring system. Fields include alert identifier, room identifier, triggered occupancy count, configured maximum capacity, alert state (Fired or Resolved), notification dispatch timestamp, and resolution timestamp. This table supports the alert history view in the administrator dashboard and provides an audit trail for compliance reporting.

APPENDIX B – API ENDPOINT REFERENCE

The SentryDash backend exposes a RESTful API built with Node.js and Express.js. All endpoints are prefixed with /api/v1 and require a valid JWT Bearer token in the Authorization header unless otherwise stated. The following tables document the core endpoint categories.

Authentication Endpoints

POST /api/v1/auth/login: Accepts institutional email and password. Returns a signed JWT token containing user_id, role, and expiry. No authentication required. Rate-limited to 10 requests per minute per IP address to prevent brute-force attacks.

POST /api/v1/auth/logout: Invalidates the current session token. Requires a valid JWT. Returns HTTP 200 on success.

GET /api/v1/auth/me: Returns the authenticated user profile including name, role, and department. Requires a valid JWT.

Room Endpoints

GET /api/v1/rooms: Returns a paginated list of all active rooms with current occupancy counts, capacity, room type, and amenities. Supports query parameters for filtering by type (theory, lab) and searching by room name or identifier. Optional authentication for public dashboard support.

GET /api/v1/rooms/:roomId: Returns detailed information for a specific room including full metadata, current occupancy, today's schedule, and recent occupancy trend. Requires valid JWT for full data access.

POST /api/v1/rooms/:roomId/reserve: Creates a reservation for a room. Validates against schedule conflicts and capacity constraints. Returns reservation confirmation or a conflict error with suggested alternatives. Requires faculty or admin role.

Occupancy Endpoints

POST /api/v1/occupancy/entry: Records a student entry event for the specified room. Validates room capacity against current count. If the room is at capacity, returns HTTP 409 Conflict with recommended alternative rooms. Triggers an Azure Monitor custom metric push for threshold evaluation.

POST /api/v1/occupancy/exit: Records a student exit event. Decrements the live occupancy count for the specified room. If the room was previously in an alert state, this event may trigger alert resolution.

GET /api/v1/occupancy/:roomId/history: Returns historical occupancy data for a room over a configurable time window. Supports aggregation by hour or day. Used to populate trend charts in the administrator dashboard.

Alert Endpoints

GET /api/v1/alerts: Returns all active and recently resolved alerts across the campus. Supports filtering by room, building, and alert state. Requires admin role. Paginated in descending order of trigger timestamp.

GET /api/v1/alerts/:alertId: Returns full detail for a specific alert including room metadata, breach count, resolution time, and notification dispatch records. Requires admin or faculty role.

APPENDIX C – SYSTEM SCREENSHOTS

This appendix provides annotated screenshots of the SentryDash platform as deployed, illustrating the user experience across all three stakeholder roles. Each screenshot demonstrates a specific platform capability corresponding to the MVP requirements defined in Chapter 3.

C.1 Student Dashboard

Figure C1 shows the student-facing room listing view accessible after login as Alice Johnson (Student role). The interface displays all available rooms organized by type — All Rooms, Theory Rooms, and Labs — and shows real-time occupancy data including current occupant count, total capacity, and a visual capacity progress bar. Room A101, Lecture Hall A101, is displayed with 4 out of 60 occupants at 7% capacity. Available amenities including WiFi and Smart Board are listed below the capacity bar. The Reserve and View Details action buttons are accessible for each room listing.

C.2 Teacher Dashboard

Figure C2 shows the faculty-facing view for Prof. Frank Miller (Teacher role). The interface is identical in structure to the student view but with additional reservation and management controls enabled by the elevated role. Faculty members can search for rooms by name or identifier, filter by room type, and directly initiate room reservations. The Teacher role badge is displayed in the header alongside the logout option, and all write-enabled actions are available in this view.

C.3 Room Detail and Occupancy View

Figure C3 shows the room detail view for Lecture Hall A101. The view presents a comprehensive summary of the room's current state, including the live occupancy bar showing 4 of 60 occupants at 7% capacity. The Room Details panel displays capacity (60 people), internet availability, Smart Board availability, and GPS location coordinates (10, 20). The Schedule panel shows the day's timetabled sessions: CS101 from 09:00 to 10:30 and MA102 from 11:00 to 12:30. The Mark Entry and Mark Exit action buttons allow students to register their presence, which updates the live occupancy count in real time.

C.4 Reservation Confirmation and Alert System

Figure C4 demonstrates the reservation confirmation flow for CS101 Section A. Upon successful reservation, a green confirmation banner is displayed with the message confirming room assignment. An amber warning panel below reports that two students in the room are not enrolled in CS101: Bob Smith and Ethan Brown. This non-enrollment detection feature enhances institutional compliance by alerting faculty to unauthorized occupants. A blue suggestion panel at the bottom recommends Computer Lab A102 as an alternate room with 29 available seats and a calculated distance of 7.07 units from the current location, enabling faculty to proactively manage overflow situations.

APPENDIX D – AZURE CONFIGURATION DETAILS

This appendix documents the Azure resource configuration used during the SentryDash development and testing phase. All resources are provisioned within a single resource group (rg-sentrydash-dev) in the East Asia region under the Azure Student Subscription.

D.1 Azure App Service Configuration

The Node.js backend is hosted on an Azure App Service instance using the F1 Free Tier App Service Plan. Node.js runtime version 18 LTS is configured as the runtime stack. Application settings are used to inject the database connection string (DB_CONNECTION_STRING), Application Insights instrumentation key (APPINSIGHTS_INSTRUMENTATIONKEY), JWT signing secret (JWT_SECRET), and CORS allowed origin (FRONTEND_URL) as environment variables. Always On is disabled under the F1 tier; the backend is warmed up by the CI/CD health check step after each deployment.

D.2 Azure SQL Database Configuration

The database is provisioned as an Azure SQL Database Single Database using the S0 service tier with 250MB maximum storage. The logical SQL server is configured with SQL authentication only during the development phase, with Microsoft Entra ID authentication planned for production hardening. The database firewall allows all Azure services and the developer IP range. SSL enforcement is enabled, and Transparent Data Encryption is active by default. The database collation is set to SQL_Latin1_General_CP1_CI_AS to ensure compatibility with standard alphanumeric queries.

D.3 Application Insights Configuration

Application Insights is provisioned as a Classic resource type with a Log Analytics Workspace backend. The Node.js Application Insights SDK (version 3.x) is initialized at application startup using the APPINSIGHTS_INSTRUMENTATIONKEY environment variable. Custom event tracking is configured for room entry events, alert trigger events, and room recommendation requests. Sampling is set to 100% during development to ensure all telemetry is captured for analysis. Retention is set to 90 days.

D.4 Azure Static Web Apps Configuration

The React.js frontend is deployed to Azure Static Web Apps using the Free tier. The deployment is connected directly to the GitHub repository main branch, with GitHub Actions automatically triggering a new build and deploy on each push. The build configuration specifies the frontend directory as the app location and dist as the output location. Custom routing rules are configured in the staticwebapp.config.json file to redirect all unknown routes to index.html, enabling client-side routing via React Router. SSL is managed automatically by Azure Static Web Apps at no additional cost.

D.5 Azure Logic Apps Workflow

The alert notification workflow is implemented as a Consumption plan Logic App with an Azure Monitor alert trigger as the entry point. When a monitor alert fires, the workflow reads the alert payload to extract the room identifier, current occupancy count, and maximum capacity. It then executes a Condition block that branches based on alert state: Fired or Resolved. For Fired state, the workflow dispatches an email notification via the Office 365 connector to the campus administrator distribution list and an SMS via the Twilio connector to the assigned faculty member. For Resolved state, a resolution notification is dispatched to confirm that the situation has normalized. Run history is retained for 30 days, providing an audit trail of all notifications sent.

APPENDIX E – GITHUB ACTIONS CI/CD WORKFLOW SUMMARY

The SentryDash CI/CD pipeline is implemented using GitHub Actions and consists of two primary workflow files: `main-deploy.yml` for the main development branch and a `production-release.yml` for tagged release deployments. The following subsections describe the key stages of the main deployment workflow.

E.1 Workflow Trigger Configuration

The `main-deploy.yml` workflow is triggered on push events to the main branch and on manual workflow dispatch. Pull request events do not trigger a full deployment; instead, they trigger a `lint-and-test` job that runs ESLint, unit tests, and a database migration dry-run to validate correctness before merge. This separation ensures that the main branch always represents a deployable state.

E.2 Backend Deployment Stage

The backend deployment stage runs on `ubuntu-latest` and executes the following steps in sequence: repository checkout, Node.js 18 LTS setup with npm cache configuration, dependency installation via `npm ci`, unit test execution via `npm test` with coverage reporting to GitHub Annotations, Azure login using the service principal credentials stored as GitHub Secrets, deployment to Azure App Service using the `azure/webapps-deploy` action with the `publish` profile secret, and finally a health check using `curl` against the `/health` endpoint with a 30-second retry loop to confirm successful startup before proceeding.

E.3 Frontend Deployment Stage

The frontend deployment stage depends on the backend stage and runs only after it completes successfully. It executes the following steps: repository checkout, Node.js 18 LTS setup, dependency installation via `npm ci` in the frontend directory, frontend build with the `VITE_API_URL` environment variable set to the App Service URL output from the backend stage, and deployment to Azure Static Web Apps using the `Azure/static-web-apps-deploy` action with the deployment token stored as a GitHub Secret. The step reports the deployed URL in the GitHub Actions summary for quick access.

E.4 Rollback Procedure

In the event that a deployment introduces a regression, the rollback procedure involves re-running the workflow on the last known-good commit SHA using the manual workflow dispatch trigger with an input parameter specifying the target commit. The backend stage will deploy the specified commit to App Service using a deployment slot swap pattern, ensuring zero-downtime rollback. The previous deployment state is retained in the App Service deployment history for 10 releases, allowing rollback to any of the last ten releases within minutes.

5.5 TESTING STRATEGY

The SentryDash testing strategy is organized across three levels to ensure correctness at the unit, integration, and end-to-end layers of the application stack.

Unit Testing

Unit tests are written using the Jest testing framework for the Node.js backend. Each Express.js route handler and service module is tested in isolation using mock dependencies for the database and Azure SDK clients. Test coverage targets include authentication logic (JWT generation and validation), occupancy counting logic (entry and exit event processing), alert threshold evaluation logic, and room recommendation ranking algorithm. The unit test suite is executed on every pull request as part of the GitHub Actions CI workflow, with coverage reports published to GitHub Annotations. A minimum coverage threshold of 70% is enforced on core business logic modules.

Integration Testing

Integration tests validate the interaction between the backend API layer and the Azure SQL Database using a dedicated test database instance provisioned with an identical schema. Integration tests cover the complete request lifecycle for all core API endpoints, including authentication middleware, database query execution, and response serialization. These tests are executed in the CI pipeline on pull requests targeting the main branch and use Supertest to make HTTP requests against a running Express.js server instance connected to the test database.

End-to-End Testing

End-to-end tests are implemented using Playwright and validate user-facing workflows across the full stack from the React.js frontend through the backend API to the database. Key scenarios covered include student login and room listing view, student room entry and live occupancy count update, faculty room reservation and non-enrolled student detection alert, and administrator alert dashboard and resolution flow. These tests run in the CI pipeline on merges to the main branch against the staging deployment environment, providing full confidence that each release is functionally correct before it reaches production.

5.6 SECURITY AUDIT FINDINGS

A static security audit was conducted on the SentryDash codebase prior to the first production deployment. The audit used ESLint with the eslint-plugin-security ruleset for the frontend and backend JavaScript code, and the npm audit command to scan all third-party dependencies for known vulnerabilities. The following findings and resolutions were identified:

- **Finding 1:** Hardcoded Credentials Risk: An early development version of the backend contained a hardcoded database connection string in the server initialization file. This was resolved by migrating all sensitive values to Azure App Service application settings and enforcing a pre-commit hook that scans for credential patterns in staged files.

- **Finding 2:** Insecure JWT Secret: The initial JWT signing secret was a short, guessable string. This was resolved by replacing it with a 256-bit randomly generated secret stored as a GitHub Secret and injected at build time.
- **Finding 3:** Missing Rate Limiting: The authentication endpoint lacked rate limiting, exposing it to brute-force attacks. This was resolved by integrating the express-rate-limit middleware with a window of 15 minutes and a maximum of 10 login attempts per IP address.
- **Finding 4:** Dependency Vulnerabilities: npm audit identified three moderate-severity vulnerabilities in transitive dependencies. All three were resolved by upgrading the affected packages to their patched versions. The CI pipeline is configured to run npm audit on every build and fail the pipeline if any high or critical vulnerabilities are detected.
- **Finding 5:** CORS Misconfiguration: An early version of the CORS configuration allowed all origins by setting the origin parameter to a wildcard. This was resolved by restricting CORS to the exact Azure Static Web Apps domain, preventing unauthorized cross-origin requests from untrusted domains.

GLOSSARY OF TERMS

The following terms are used throughout this report and are defined here for clarity:

API (Application Programming Interface): A set of defined rules that enable different software applications to communicate with each other. In SentryDash, the Node.js backend exposes a RESTful API that the React.js frontend consumes.

Azure Monitor: A comprehensive monitoring service in Microsoft Azure that collects, analyzes, and acts on telemetry from cloud and on-premises environments, used in SentryDash as the core observability layer.

Application Insights: An Application Performance Management (APM) service within Azure Monitor that provides request-level performance tracing, exception logging, and custom event telemetry for web applications.

Azure App Service: A fully managed Platform-as-a-Service (PaaS) for hosting web applications, REST APIs, and mobile backends on Microsoft Azure without managing the underlying infrastructure.

Azure SQL Database: A fully managed relational database service based on the Microsoft SQL Server engine, providing automatic backups, high availability, and elastic scaling in the Azure cloud.

Azure Logic Apps: A serverless workflow automation service in Azure that allows the creation of automated pipelines connecting services, data sources, and APIs without writing code.

Azure Functions: A serverless compute service in Azure that allows execution of small units of code (functions) in response to events without provisioning or managing servers.

Azure Static Web Apps: A service that automatically builds and deploys full-stack web apps to Azure from a GitHub or Azure DevOps repository, with integrated CDN and SSL management.

CI/CD (Continuous Integration/Continuous Deployment): A software development practice that automates the building, testing, and deployment of applications whenever code changes are committed to a source repository.

CORS (Cross-Origin Resource Sharing): An HTTP-header-based mechanism that controls which origins (domains) are permitted to make requests to a web server, preventing unauthorized cross-site requests.

JWT (JSON Web Token): A compact, URL-safe token format used to securely transmit information between parties. In SentryDash, JWTs carry user identity and role claims for authentication and authorization.

KQL (Kusto Query Language): A read-only query language used to analyze data in Azure Monitor Log Analytics workspaces, allowing powerful queries across logs and telemetry data.

Microservice Architecture: A software design pattern that structures an application as a collection of small, independent services that communicate over APIs, enabling independent deployment and scaling of each service.

MVP (Minimum Viable Product): The simplest version of a product that delivers enough value to satisfy early users and provide feedback for future development without building unnecessary features.

RBAC (Role-Based Access Control): An approach to restricting system access where permissions are assigned to roles rather than individual users, and users are assigned to roles based on their responsibilities.

REST (Representational State Transfer): An architectural style for designing networked applications that uses HTTP methods (GET, POST, PUT, DELETE) to perform CRUD operations on resources identified by URLs.

SLA (Service Level Agreement): A commitment between a service provider and a client that defines the level of service expected, including uptime guarantees, performance benchmarks, and support response times.

Telemetry: The automated collection and transmission of data from remote sources, used in SentryDash to refer to metrics, logs, and traces collected from Azure services and the application layer.